

Numerical Methods Lab

Lab 19 – 26 | C Programs

C Source Code

Lab-19: Jacobi Iteration Method

Question: Write a program for Jacobi Iteration Method. $4x + y + 2z = 4$, $3x + 5y + z = 7$, $x + y + 3z = 3$

C Program:

```
#include <stdio.h>
#include <math.h>
#define MAX_ITER 100

int main() {
    float a[3][4], x[3] = {0}, xn[3], error;
    int i, j, iter;

    printf("Enter the coefficients for the system of equations (ax + by + cz = d):\n");
    char vars[] = "xyz";
    for (i = 0; i < 3; i++) {
        printf("\nEquation %d:\n", i+1);
        printf(" Coefficient of x (a%d): ", i+1); scanf("%f", &a[i][0]);
        printf(" Coefficient of y (b%d): ", i+1); scanf("%f", &a[i][1]);
        printf(" Coefficient of z (c%d): ", i+1); scanf("%f", &a[i][2]);
        printf(" Constant term (d%d): ", i+1); scanf("%f", &a[i][3]);
    }

    printf("\n%-5s | %-10s | %-10s | %-10s | %-15s\n", "Iter", "x", "y", "z", "Max Error");
    printf("-----\n");

    for (iter = 1; iter <= MAX_ITER; iter++) {
        xn[0] = (a[0][3] - a[0][1]*x[1] - a[0][2]*x[2]) / a[0][0];
        xn[1] = (a[1][3] - a[1][0]*x[0] - a[1][2]*x[2]) / a[1][1];
        xn[2] = (a[2][3] - a[2][0]*x[0] - a[2][1]*x[1]) / a[2][2];
        error = 0;
        for (j = 0; j < 3; j++) {
            float e = fabs(xn[j] - x[j]);
            if (e > error) error = e;
            x[j] = xn[j];
        }
        printf("%-5d | %-10.6f | %-10.6f | %-10.6f | %-15.6e\n", iter, x[0], x[1], x[2], error);
        if (error < 1e-6) {
            printf("\nConverged in %d iterations.\n", iter);
            printf("Final Solution: x=%.6f, y=%.6f, z=%.6f\n", x[0], x[1], x[2]);
            return 0;
        }
    }
    printf("\nDid not converge within max iterations.\n");
    return 0;
}
```

Lab-20: Gauss-Seidel Method

Question: Write a program for Gauss-Seidel Method. $4x + y + 2z = 4$, $3x + 5y + z = 7$, $x + y + 3z = 3$

C Program:

```
#include <stdio.h>
#include <math.h>
#define MAX_ITER 100

int main() {
    float a[3][4], x[3] = {0}, x_old[3], error;
    int i, iter;

    printf("Enter the coefficients for the system (ax + by + cz = d):\n");
    for (i = 0; i < 3; i++) {
        printf("\nEquation %d:\n", i+1);
        printf(" Coefficient of x (a%d): ", i+1); scanf("%f", &a[i][0]);
        printf(" Coefficient of y (b%d): ", i+1); scanf("%f", &a[i][1]);
        printf(" Coefficient of z (c%d): ", i+1); scanf("%f", &a[i][2]);
        printf(" Constant term (d%d): ", i+1); scanf("%f", &a[i][3]);
    }

    printf("\n%-5s | %-10s | %-10s | %-10s | %-15s\n", "Iter", "x", "y", "z", "Max Error");
    printf("-----\n");

    for (iter = 1; iter <= MAX_ITER; iter++) {
        for (i = 0; i < 3; i++) x_old[i] = x[i];
        x[0] = (a[0][3] - a[0][1]*x[1] - a[0][2]*x[2]) / a[0][0];
        x[1] = (a[1][3] - a[1][0]*x[0] - a[1][2]*x[2]) / a[1][1];
        x[2] = (a[2][3] - a[2][0]*x[0] - a[2][1]*x[1]) / a[2][2];
        error = 0;
        for (i = 0; i < 3; i++) {
            float e = fabs(x[i] - x_old[i]);
            if (e > error) error = e;
        }
        printf("%-5d | %-10.6f | %-10.6f | %-10.6f | %-15.6e\n", iter, x[0], x[1], x[2], error);
        if (error < 1e-6) {
            printf("\nConverged in %d iterations.\n", iter);
            printf("Final Solution: x=%.6f, y=%.6f, z=%.6f\n", x[0], x[1], x[2]);
            return 0;
        }
    }
    printf("\nDid not converge.\n");
    return 0;
}
```

Lab-21: Taylor Series Method

Question: Write a program for Taylor series expansion. $dy/dx = 2y/x$, $x_0=4$, $y_0=6$, $h=0.25$, evaluate at $x=6$

C Program:

```
#include <stdio.h>
#include <math.h>

int main() {
    float x0, y0, xn, h, x, y, f1, f2, f3;
    int i, steps;

    printf("Taylor Series Method for dy/dx = 2y/x\n");
    printf("-----\n");
    printf("Enter initial x (x0): "); scanf("%f", &x0);
    printf("Enter initial y (y0): "); scanf("%f", &y0);
    printf("Enter target x value: "); scanf("%f", &xn);
    printf("Enter step size (h): "); scanf("%f", &h);

    x = x0; y = y0;
    steps = (int)((xn - x0) / h + 0.5);

    printf("\n%-5s | %-10s | %-10s\n", "Step", "x", "y");
    printf("-----\n");
    printf("%-5d | %-10.4f | %-10.4f\n", 0, x, y);

    for (i = 1; i <= steps; i++) {
        f1 = 2*y/x;
        f2 = 2*(x*f1 - y)/(x*x);
        f3 = 2*(x*x*f2 - 2*x*f1 + 2*y)/(x*x*x);
        y = y + h*f1 + (h*h/2)*f2 + (h*h*h/6)*f3;
        x = x + h;
        printf("%-5d | %-10.4f | %-10.4f\n", i, x, y);
    }
    printf("-----\n");
    printf("Approximate solution at x = %.1f is y = %.6f\n", xn, y);
    return 0;
}
```

Lab-22: Euler's Method

Question: Write a program for Euler's method. $dy/dx = 2y/x$, $x_0=4$, $y_0=6$, $h=0.25$, evaluate at $x=6$

C Program:

```
#include <stdio.h>

float f(float x, float y) { return 2*(y/x); }

int main() {
    float x0, y0, xn, h, x, y;
    int i, steps;

    printf("Euler's Method Solver for dy/dx = 2y/x\n");
    printf("-----\n");
    printf("Enter initial x (x0): "); scanf("%f", &x0);
    printf("Enter initial y (y0): "); scanf("%f", &y0);
    printf("Enter target x value to solve for: "); scanf("%f", &xn);
    printf("Enter step size (h): "); scanf("%f", &h);

    x = x0; y = y0;
    steps = (int)((xn - x0) / h + 0.5);

    printf("\n%-5s | %-10s | %-10s\n", "Step", "x", "y");
    printf("-----\n");
    printf("%-5d | %-10.4f | %-10.4f\n", 0, x, y);

    for (i = 1; i <= steps; i++) {
        y = y + h * f(x, y);
        x = x + h;
        printf("%-5d | %-10.4f | %-10.4f\n", i, x, y);
    }
    printf("-----\n");
    printf("Approximate solution at x = %.1f is y = %.6f\n", xn, y);
    return 0;
}
```

Lab-23: Heun's Method

Question: Write a program for Heun's method. $dy/dx = x + y$, $x_0=0$, $y_0=1$, $h=0.1$, evaluate at $x=1$

C Program:

```
#include <stdio.h>

float f(float x, float y) { return x + y; }

int main() {
    float x0, y0, xn, h, x, y, k1, k2, y_pred, slope_avg;
    int i, steps;

    printf("Heun's Method Solver for dy/dx = x + y\n");
    printf("-----\n");
    printf("Enter initial x (x0): "); scanf("%f", &x0);
    printf("Enter initial y (y0): "); scanf("%f", &y0);
    printf("Enter target x value: "); scanf("%f", &xn);
    printf("Enter step size (h): "); scanf("%f", &h);

    x = x0; y = y0;
    steps = (int)((xn - x0) / h + 0.5);

    printf("\n%-5s | %-10s | %-10s | %-10s\n", "Step", "x", "y", "Slope Avg");
    printf("-----\n");
    printf("%-5d | %-10.4f | %-10.4f | %-10s\n", 0, x, y, "N/A");

    for (i = 1; i <= steps; i++) {
        k1 = f(x, y);
        y_pred = y + h * k1;
        k2 = f(x + h, y_pred);
        slope_avg = (k1 + k2) / 2;
        y = y + h * slope_avg;
        x = x + h;
        printf("%-5d | %-10.4f | %-10.4f | %-10.4f\n", i, x, y, slope_avg);
    }
    printf("-----\n");
    printf("Approximate solution at x = %.1f is y = %.6f\n", xn, y);
    return 0;
}
```

Lab-24: Runge-Kutta 4th Order (RK4) Method

Question: Write a program for RK4 method. $dy/dx = 3x + y$, $x_0=2$, $y_0=4$, $h=0.1$, evaluate at $x=3$

C Program:

```
#include <stdio.h>
#include <math.h>

float f(float x, float y) { return 3*x + y; }

int main() {
    float x0, y0, xn, h, x, y, k1, k2, k3, k4;
    int i, steps;

    printf("RK4 Method Solver for dy/dx = 3x + y\n");
    printf("-----\n");
    printf("Enter initial x (x0): "); scanf("%f", &x0);
    printf("Enter initial y (y0): "); scanf("%f", &y0);
    printf("Enter target x value: "); scanf("%f", &xn);
    printf("Enter step size (h): "); scanf("%f", &h);

    x = x0; y = y0;
    steps = (int)(fabs(xn - x0) / h + 0.5);

    printf("\n%-5s | %-10s | %-10s\n", "Step", "x", "y");
    printf("-----\n");
    printf("%-5d | %-10.4f | %-10.4f\n", 0, x, y);

    for (i = 1; i <= steps; i++) {
        k1 = h * f(x, y);
        k2 = h * f(x + h/2, y + k1/2);
        k3 = h * f(x + h/2, y + k2/2);
        k4 = h * f(x + h, y + k3);
        y = y + (k1 + 2*k2 + 2*k3 + k4) / 6;
        x = x + h;
        printf("%-5d | %-10.4f | %-10.4f\n", i, x, y);
    }
    printf("-----\n");
    printf("Approximate solution at x = %.1f is y = %.6f\n", xn, y);
    return 0;
}
```

Lab-25: Laplace Equation

Question: Write a program for Laplace equation using finite difference method.

C Program:

```
#include <stdio.h>
#include <math.h>
#define MAX 100
#define TOL 1e-6

int main() {
    float grid[MAX][MAX] = {0};
    int n, i, j;
    float error, old;

    printf("Enter the size of the grid (n x n): ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        grid[0][i] = 1.0;
        grid[n-1][i] = 0.0;
        grid[i][0] = 0.0;
        grid[i][n-1] = 1.0;
    }

    do {
        error = 0;
        for (i = 1; i < n-1; i++)
            for (j = 1; j < n-1; j++) {
                old = grid[i][j];
                grid[i][j] = 0.25*(grid[i-1][j]+grid[i+1][j]+grid[i][j-1]+grid[i][j+1]);
                float e = fabs(grid[i][j] - old);
                if (e > error) error = e;
            }
    } while (error > TOL);

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("%.4f ", grid[i][j]);
        printf("\n");
    }
    return 0;
}
```


Lab-26: Poisson Equation

Question: Write a program for Poisson equation using finite difference method.

C Program:

```
#include <stdio.h>
#include <math.h>
#define MAX 100

float f(float x, float y) { return 2*x*x*y*y; }

int main() {
    float grid[MAX][MAX] = {0};
    float length, width, acc, left, right, bottom, upper;
    int n, i, j;
    float error, old;

    printf("Enter length of the plate: "); scanf("%f", &length);
    printf("Enter width of the plate: "); scanf("%f", &width);
    printf("Enter the size of the grid: "); scanf("%d", &n);
    printf("Enter boundary temperatures (left, right, bottom, upper): ");
    scanf("%f %f %f %f", &left, &right, &bottom, &upper);
    printf("Enter accuracy limit: "); scanf("%f", &acc);

    for (i = 0; i < n; i++) {
        grid[0][i] = upper;
        grid[n-1][i] = bottom;
        grid[i][0] = left;
        grid[i][n-1] = right;
    }

    do {
        error = 0;
        for (i = 1; i < n-1; i++)
            for (j = 1; j < n-1; j++) {
                old = grid[i][j];
                grid[i][j] = 0.25*(grid[i-1][j]+grid[i+1][j]+grid[i][j-1]+grid[i][j+1]
                    - f((float)i/(n-1), (float)j/(n-1)));
                float e = fabs(grid[i][j] - old);
                if (e > error) error = e;
            }
    } while (error > acc);

    printf("\nDimensions: Length = %.2f, Width = %.2f\n", length, width);
    printf("Solution (Temperatures):\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("%8.4f ", grid[i][j]);
        printf("\n");
    }
    return 0;
}
```